

Universidade Federal de Viçosa

Campus **Rio Paranaíba**

“Pedro Augusto Londe Ribeiro” – “9340”

Fernando Gomes Sanho Furtado – “9348”

Gustavo Del Castillo – “9346”

“Relatório Projeto 1 - Simulação Autômato”

Rio Paranaíba - MG

2026

Universidade Federal de Viçosa
Campus **Rio Paranaíba**

Pedro Augusto Londe Ribeiro – 9340
Fernando Gomes Sanho Furtado – 9348
Gustavo Del Castillo - 9346

“Relatório Projeto 1 - Simulação Autômato”

Trabalho realizado para obtenção de nota
na disciplina SIN 131 - Introdução à Teoria
da Computação da Universidade Federal de
Viçosa - Campus de Rio Paranaíba.

Rio Paranaíba - MG

2026

RESUMO

Este trabalho tem como finalidade a implementação de uma aplicação voltada à Teoria da Computação, com foco em Autômatos Finitos Determinísticos, Autômatos Finitos Não Determinísticos e Gramáticas Regulares. O sistema permite a visualização de autômatos por meio de uma interface gráfica, a simulação da aceitação de palavras e a realização de conversões entre Gramáticas Regulares e Autômatos Finitos. Dessa forma, o projeto reúne funcionalidades que auxiliam na identificação e na compreensão dos temas abordados.

Sumário

1	Introdução	2
2	Estrutura do Projeto	2
3	Tecnologias Utilizadas	2
4	Requisitos e Funcionalidades do Sistema	3
4.1	Requisitos Funcionais	3
5	Descrição das Telas	4
5.1	Telas de Cadastro	4
5.2	Tela de Operações e Visualização	4
5.3	Imagens	4
6	Diagrama	5
7	Potencialidades e Limitações	7
7.1	Potencialidades	7
7.2	Limitações	7
8	Conclusão	7

1 Introdução

A Teoria da Computação estuda modelos utilizados para representar linguagens e reconhecer padrões. Entre seus principais conceitos estão os Autômatos Finitos e as Gramáticas Regulares. Neste contexto, o projeto propõe uma aplicação capaz de cadastrar, visualizar, converter e minimizar autômatos, além de simular a aceitação de palavras por meio de uma interface gráfica.

2 Estrutura do Projeto

O projeto foi desenvolvido de forma modular e orientada a objetos, distribuindo suas funcionalidades em arquivos distintos para facilitar a organização, a manutenção e futuras alterações.

O arquivo `automato.py` contém a classe `Automato`, responsável por representar os autômatos, processar palavras e calcular o fecho- ϵ . Já o arquivo `gramatica.py` contém a classe `GramaticaRegular`, utilizada para representar e validar Gramáticas Regulares.

Os módulos `conversao_gr_af.py`, `conversao_af_gr.py` e `conversao_afn_afd.py` realizam as conversões entre gramáticas e autômatos, enquanto o arquivo `minimizacao_afd.py` executa a minimização de AFDs. A interação com o sistema ocorre pelo terminal, por meio do arquivo `main.py`, ou pela interface gráfica em PyQt5, implementada em `main_gui.py`.

3 Tecnologias Utilizadas

O desenvolvimento do projeto utilizou tecnologias voltadas para a implementação de algoritmos, criação da interface gráfica, visualização de autômatos e controle de versão, destacando-se:

- **Python:** linguagem principal utilizada na implementação das classes, dos algoritmos de conversão, da minimização e da simulação de palavras;
- **PyQt5:** biblioteca utilizada no desenvolvimento da interface gráfica, permitindo o cadastro dos dados e a execução das operações do sistema;

- **NetworkX:** biblioteca empregada na representação dos estados e das transições dos autômatos por meio de grafos;
- **Matplotlib:** biblioteca utilizada para apresentar graficamente os autômatos construídos e os resultados das conversões;
- **Orientação a Objetos:** empregado na organização das classes *Automato* e *GramaticaRegular*, facilitando a reutilização e a manutenção do código;
- **GitHub:** ferramentas utilizadas para o versionamento, armazenamento e compartilhamento do projeto entre os integrantes do grupo. “

4 Requisitos e Funcionalidades do Sistema

O sistema foi desenvolvido para funcionar como um simulador de autômatos. As funcionalidades implementadas buscam atender aos principais requisitos definidos para o projeto.

4.1 Requisitos Funcionais

- Permitir o cadastro de Autômatos Finitos Não Determinísticos
- Converter um AFN em um AFD equivalente;
- Simular o processamento de palavras em um AFD, indicando se foram aceitas ou rejeitadas;
- Permitir o cadastro e a validação de Gramáticas Regulares;
- Converter uma Gramática Regular em um AFN correspondente;
- Converter um Autômato Finito em uma Gramática Regular;
- Minimizar AFDs por meio da remoção de estados inalcançáveis e equivalentes;
- Exibir graficamente os estados e as transições dos autômatos;
- Identificar entradas inválidas e informar o usuário. “

5 Descrição das Telas

A interface foi desenvolvida em PyQt5 com tema escuro e navegação via menu lateral, que permanece desabilitado até que uma estrutura seja cadastrada. O menu dá acesso às telas de cadastro, operações e encerramento do programa.

5.1 Telas de Cadastro

- **AFN:** Permite inserir estados, alfabeto, condições iniciais/finais e transições no formato *origem símbolo destino* (utilizando & para transições vazias). O botão *Processar e Criar AFN* gera o autômato e libera o painel de operações.
- **Gramática Regular:** Recebe os símbolos (terminais e não terminais), o elemento inicial e as regras de produção. O cadastro habilita as conversões automáticas para AFN ou AFD.

5.2 Tela de Operações e Visualização

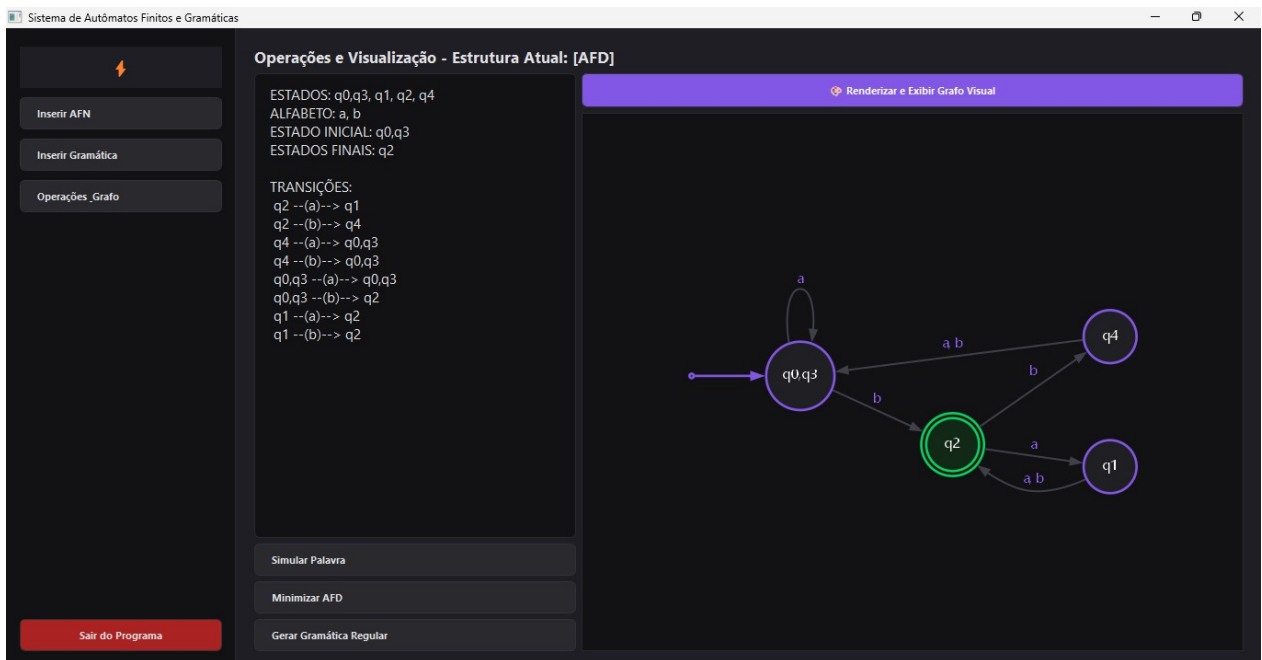
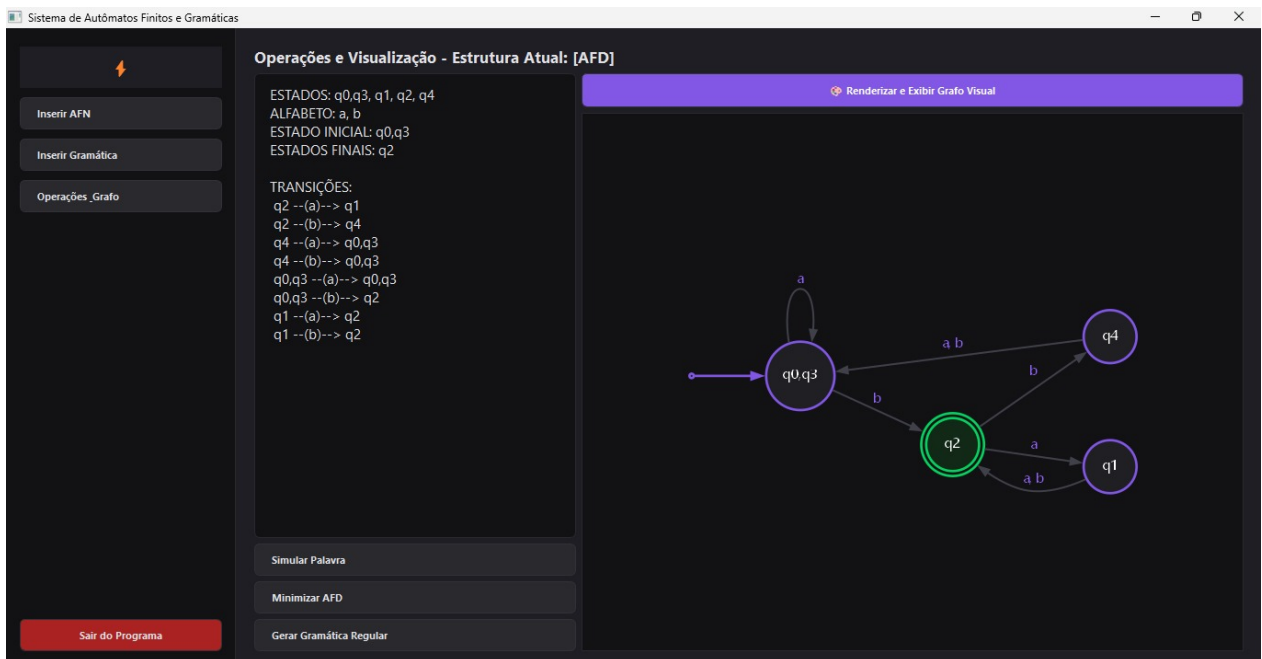
Este painel centraliza as ações compatíveis com a estrutura selecionada:

- **Para AFN:** Conversão para AFD e Gramática Regular.
- **Para AFD:** Simulação de palavras, minimização e geração de Gramática Regular.
- **Para Gramática:** Conversão direta para AFN ou AFD.

Ademais, o botão *Renderizar e Exibir Grafo Visual* gera a representação gráfica dos estados e transições de autômatos (destacando início e fim). Para gramáticas, a exibição é estritamente textual.

5.3 Imagens

A seguir imagens das telas:



6 Diagrama

A seguir, serão apresentados dois diagramas de classes. O primeiro apresenta uma visão completa, contendo as classes, seus atributos, métodos e relacionamentos. O segundo apre-

senta uma visão simplificada, destacando apenas as principais classes do sistema. As classes *Automato* e *GramaticaRegular* representam a lógica da aplicação, enquanto a classe *JanelaPrincipal* gerencia a interface gráfica e utiliza essas estruturas.

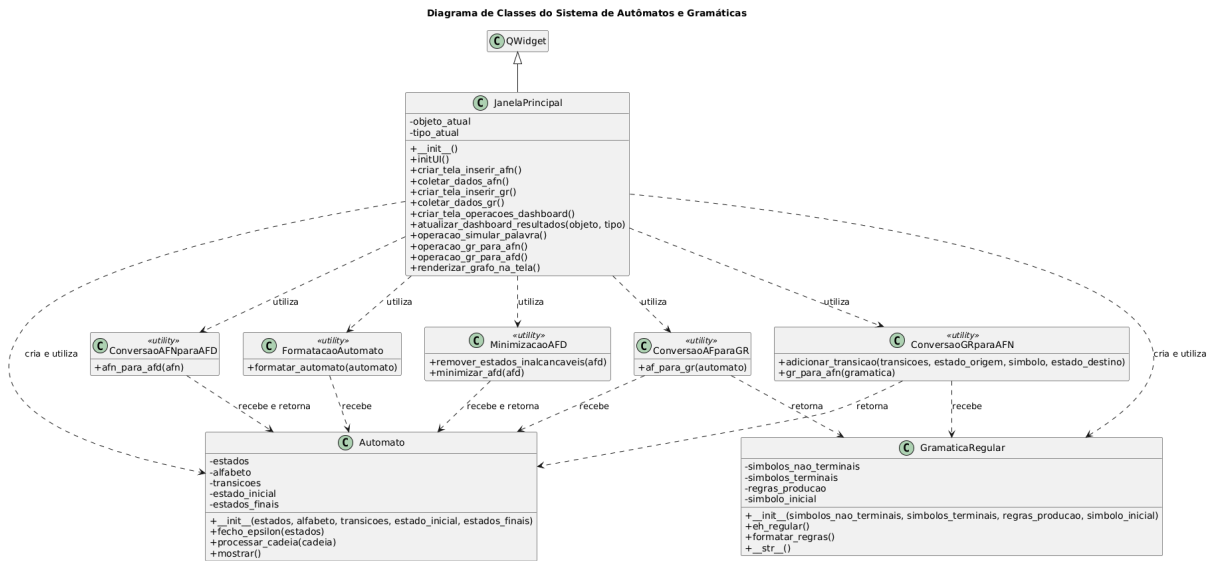


Figura 1: Diagrama de classes completo.

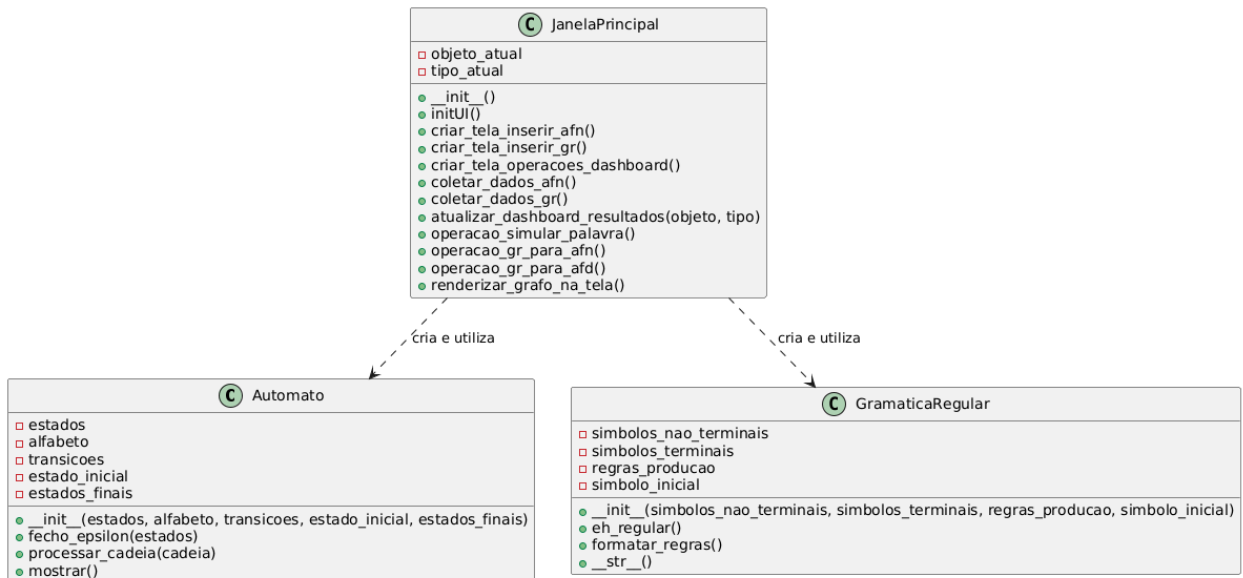


Figura 2: Diagrama de classes simplificado.

7 Potencialidades e Limitações

7.1 Potencialidades

- Permite cadastrar, converter e minimizar autômatos;
- Realiza simulações de palavras e conversões com Gramáticas Regulares;
- Possui interface gráfica e visualização dos autômatos.

7.2 Limitações

- A entrada dos dados deve seguir formatos específicos;
- A visualização gráfica depende do Graphviz;
- Os dados não são armazenados após o encerramento do programa.

8 Conclusão

O desenvolvimento do projeto permitiu aplicar, de forma prática, os principais conceitos relacionados a Autômatos Finitos e Gramáticas Regulares. O sistema possibilita cadastrar estruturas, realizar conversões, simular palavras, minimizar AFDs e visualizar os autômatos por meio de uma interface gráfica. Dessa forma, a aplicação atende aos objetivos propostos e contribui para a compreensão dos conteúdos estudados na disciplina de Teoria da Computação.